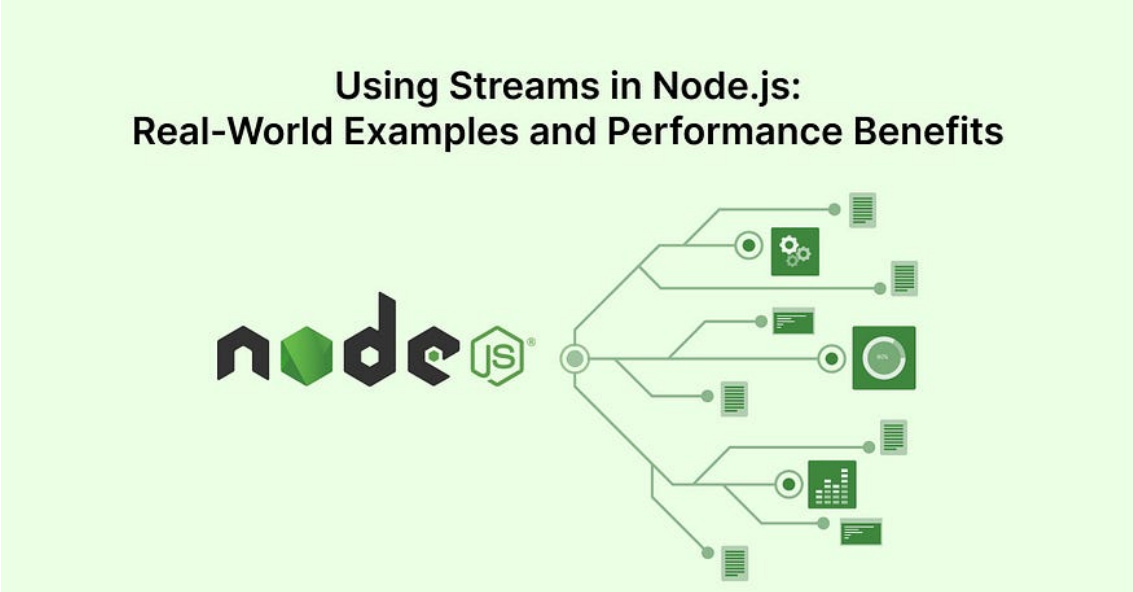
Streams

Streams are objects in Node.js that efficiently handle data flow, ***especially when dealing with large datasets.*** ***Large excel , vedio, audio, text file, csv file***



Understanding Streams

Streams are collections of data that can be **read** or **written** as **chunks,** rather than loading the entire data set into memory.

They enable ***efficient processing of data by allowing us to operate*** **on small portions of data at a time**.

- Useful when dealing with large files or network communication like ***Images*** , ***Vedios***, ***Large JS File Chunks,***

Streams can be categorized into 4 types:

- **Readable Streams** represent a **source** from which data can be read.
- **Writable Streams** represent a **destination** where data can be written.
- **Duplex Streams** can be both **readable** and **writable**.
- **Transform streams** are a special type of duplex stream that can modify or transform the data while it is being read or written.

Real-World Example

1. **Reading Files:** Streams are excellent for reading and writing files, particularly when dealing with large files.

Events
- **Open event.on('open",,()=>{})**
- **data**
- **close**
- **error**

Step 1: create Stream using **`fs module`**

Step 2: listen on event like **`error` , `data`** and **run the callback**

---------- * * * * ** * ----------

2. **Writing Files:** Streams are excellent for reading and writing files, particularly when dealing with large files.

Step 1: create Stream using **`fs module`**

Step 2 :

stream.write(chunk1)

stream.write(chunk2)

stream.write(chunk1)

3. **Reading and Writing Stream:** Streams are excellent for reading and writing files, particularly when dealing with large files.

Way 1:

Task: create `**read stream**` and read data from `**input.txt**` THEN write the same data to `**output.txt**`

Way 2: using pipeline

Conclusion

Streams are a powerful feature of Node.js that provide an efficient way to handle data flow, particularly when dealing with large datasets.

They offer practical solutions to handle file operations, network communication, data compression, and more. By leveraging streams, developers can build high-performance applications that are scalable, memory-efficient, and faster

****** ☺ HAPPY Streaming ******